

Deep Learning

Paul Bürkner based on materials of Šimon Kucharský

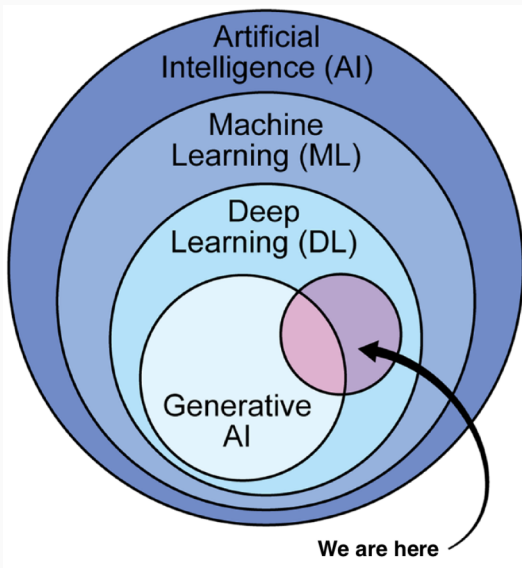


Figure 1: Shahab et al. (2024)

Deep Learning in Python

- PyTorch
- TensorFlow
- JAX
- keras

- created by Meta (formerly Facebook)
- easy to learn
- focus on research prototypes
- models are not compiled

- created by Google
- easy to learn
- focus on production
- models are compiled

- created by Google
- pure, functional approach
- JIT compiled
- fastest in runtime
- most difficult to learn

- created by Google
- API library
- Uses `PyTorch`, `TensorFlow`, or `jax` as a backend

Listing 1 Python

```
import os
os.environ["KERAS_BACKEND"] = "tensorflow" # torch, jax
import keras
```

- created by Google
- API library
- Uses `PyTorch`, `TensorFlow`, or `jax` as a backend

Listing 2 Terminal

```
conda env config vars set KERAS_BACKEND=jax
```

Listing 3 Python

```
import keras
```

- All of Deep Learning revolves around “Tensors”
- Similar to arrays in **numpy**
- Additional features:
 - Values *and gradients*
 - Can be stored on GPUs (optional)

Listing 4 Python

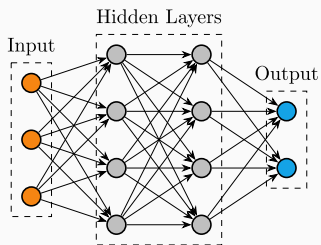
```
import os
os.environ["KERAS_BACKEND"] = "tensorflow"
import keras

x = keras.ops.zeros((16, 2))
x.shape # TensorShape([16, 2])
x.device # device:CPU:0
```

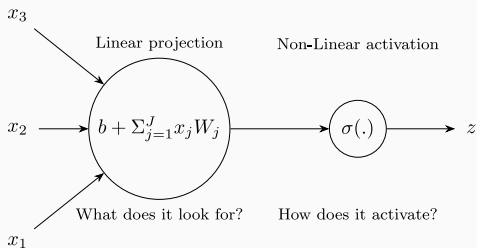
- First axis almost always **batch_size** (think “sample size”)
- Other axes contextual (**timepoint**, **feature**, **variable**, **row**, **column**, etc)

Neural networks

The anatomy of neural networks

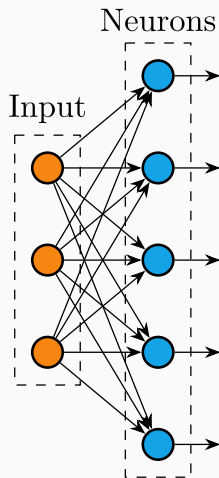


Regression + non-linear activation



Multiple regressions + non-linear activations

$$z_k = \sigma\left(b_k + \sum_{j=1}^J W_{jk} x_j\right)$$
$$z = \sigma(b + xW')$$



Listing 5 Python

```
import keras

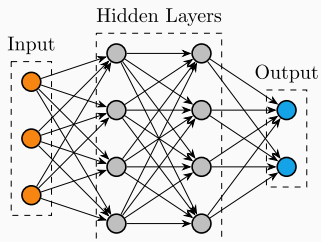
network = keras.models.Sequential([
    keras.Input((3,)),
    keras.layers.Dense(5, activation="relu"),
])
network.summary()

x = keras.random.normal((100, 3))
x.shape # TensorShape([100, 3])

z = network(x)
z.shape # TensorShape([100, 5])
```

Multi-Layer Perceptron

Multiple “layers” of perceptrons



Function composition

$$z = f(x) \text{ where } f = f_L \circ f_{L-1} \circ \dots \circ f_1$$

$$z = f_L(\dots (f_2(f_1(x))))$$

- W_{jk}^l : weight of the input j to the output k in the layer l

Multi-Layer Perceptron in Code

Listing 6 Python

```
1 import keras
2
3 network = keras.models.Sequential([
4     keras.Input((3,)),
5     keras.layers.Dense(4, activation="relu"), # 3 in, 4 out
6     keras.layers.Dense(4, activation="relu"), # 4 in, 4 out
7     keras.layers.Dense(2, activation="softmax") # 4 in, 2 out
8 ])
9 network.summary()
10
11 x = keras.random.normal((100, 3))
12 x.shape # TensorShape([100, 3])
13
14 z = network(x)
15 z.shape # TensorShape([100, 2])
```

Multi-Layer Perceptron in Code

Listing 7 Python

```
1 import keras
2
3 network = keras.models.Sequential([
4     keras.Input((3,)),
5     keras.layers.Dense(4, activation="relu"), # 3 in, 4 out
6     keras.layers.Dense(4, activation="relu"), # 4 in, 4 out
7     keras.layers.Dense(2, activation="softmax") # 4 in, 2 out
8 ])
9 network.summary()
10
11 x = keras.random.normal((100, 3))
12 x.shape # TensorShape([100, 3])
13
14 z = network(x)
15 z.shape # TensorShape([100, 2])
```

Activation functions

Why activation functions?

- A composition of linear functions is itself a linear function
- Non-linear activations introduces non-linearity
 - Represent any non-linear function
- Often used for output range control

What is an activation function?

- Basic idea: Neuron “firing activity” based on its internal state

What is an activation function?

- Basic idea: Neuron “firing activity” based on its internal state
- Requirements:

What is an activation function?

- Basic idea: Neuron “firing activity” based on its internal state
- Requirements:
 - Non-linearity (expressiveness)

What is an activation function?

- Basic idea: Neuron “firing activity” based on its internal state
- Requirements:
 - Non-linearity (expressiveness)
 - Differentiability (training)

What is an activation function?

- Basic idea: Neuron “firing activity” based on its internal state
- Requirements:
 - Non-linearity (expressiveness)
 - Differentiability (training)
 - Efficiency (scalability)

What is an activation function?

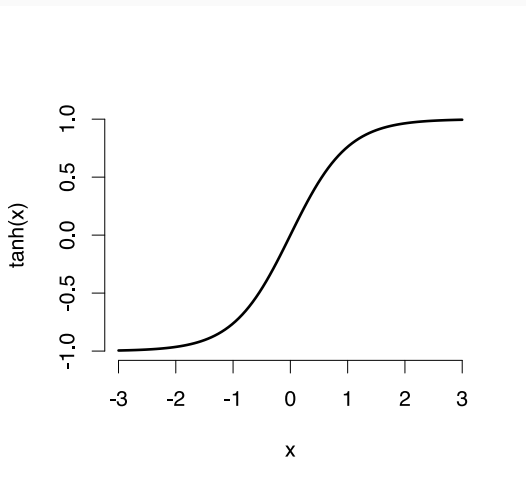
- Basic idea: Neuron “firing activity” based on its internal state
- Requirements:
 - Non-linearity (expressiveness)
 - Differentiability (training)
 - Efficiency (scalability)
- Many options (Kunc & Kléma, 2024)

What is an activation function?

- Basic idea: Neuron “firing activity” based on its internal state
- Requirements:
 - Non-linearity (expressiveness)
 - Differentiability (training)
 - Efficiency (scalability)
- Many options (Kunc & Kléma, 2024)
 - <https://keras.io/api/layers/activations/>

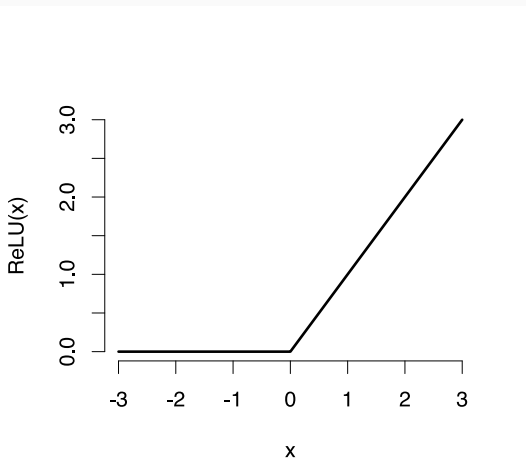
Activation functions

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



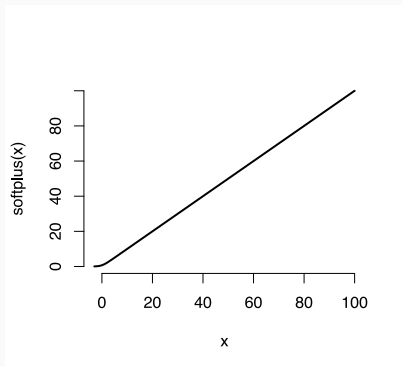
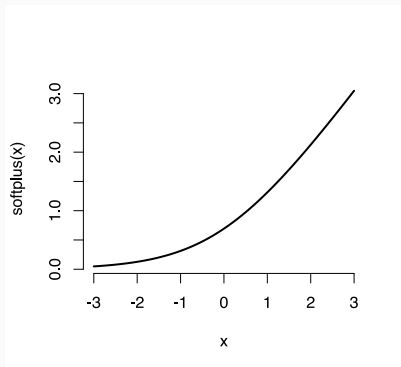
Activation functions

$$\text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$



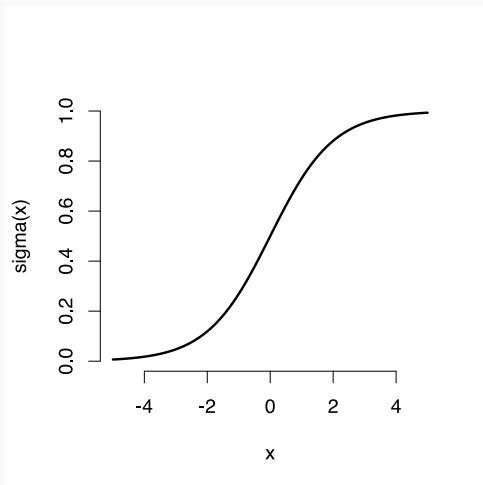
Activation functions

$$\text{softplus}(x) = \log(1 + e^x)$$



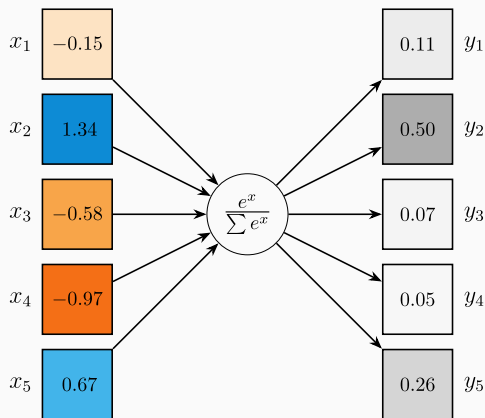
Activation functions

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Activation functions

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}}$$



Training networks

- Networks take an input and produce an output
- The output depends on the *weights* and *biases* (*parameters*) of the neurons
- *Training*: Adjusting the network parameters

1. Network

- What is the network architecture?
- What are the parameters of the network θ ?

2. Data

- What information do we have available?

3. Goal

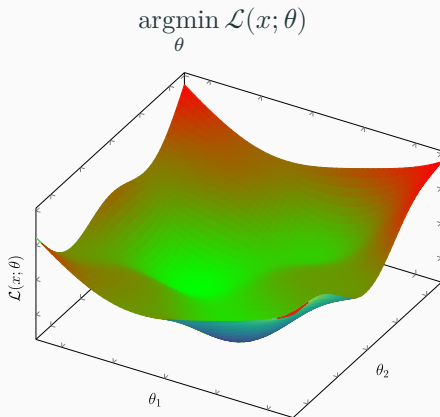
- What do we want the network to do?

- The goal is operationalized by a loss function

$$\mathcal{L}(x; \theta)$$

- θ : Network parameters
- x : Data

Minimise the loss with respect to the model parameters



Second order derivatives

- e.g., Newton's method
- Few slow steps
- Small data

First order derivatives

- e.g., Gradient descent
- Many cheap steps
- Large data

Function values & heuristics

- e.g., Nelder-Mead, BFGS, Differential evolution, ...

$$\theta_{n+1} = \theta_n - \gamma \Delta_{\theta} \mathcal{L}(x; \theta_n)$$

- θ_{n+1} : New network weights
- θ_n : Current network weights
- γ : Learning rate
- Δ_{θ} : Gradient (matrix of partial derivatives w.r.t network weights)
- $\mathcal{L}$: Loss function
- x : Data

Stochastic gradient descent (SGD)

GD

- Run through *all* data to do a single step

SGD

- Make a single step based on a *subset* of the data (minibatch)

$$\theta_{n+1} = \theta_n - \gamma \Delta_{\theta} \mathcal{L}(x; \theta_n)$$

- Too small LR: Too many steps to converge
- Too large LR: May not converge

- Adjust LR based on multiple iterations
- Individual LR per parameter

$$g_n = \Delta_{\theta} \mathcal{L}(x; \theta_n)$$

$$G_n = G_{n-1} + g_n^2$$

$$\theta_{n+1} = \theta_n + \frac{\gamma}{\sqrt{G_n} + \epsilon} g_n$$

- Accumulate gradient over iterations
- Smoother parameter updates
- Avoid getting stuck in local minima, saddle points

$$m_n = \beta m_{n-1} + (1 - \beta) \Delta_{\theta} \mathcal{L}(x; \theta_n)$$

$$\theta_{n+1} = \theta_n - \gamma m_n$$

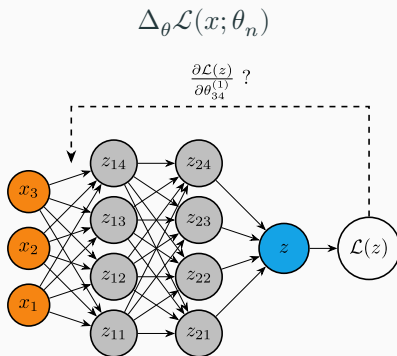
$$g_n = \Delta_{\theta} \mathcal{L}(x; \theta_n)$$

$$m_n = \beta_1 m_{n-1} + (1 - \beta_1) g_n; \quad \hat{m}_n = \frac{m_n}{1 - \beta_1^n}$$

$$v_n = \beta_2 v_{n-1} + (1 - \beta_2) g_n^2; \quad \hat{v}_n = \frac{v_n}{1 - \beta_1^n}$$

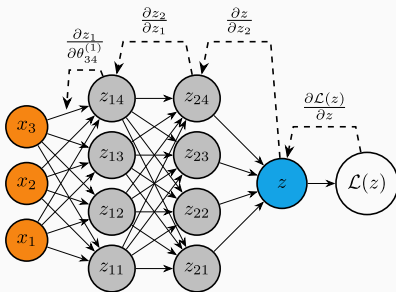
$$\theta_{n+1} = \theta_n - \frac{\gamma}{\sqrt{\hat{v}_n} + \epsilon} \hat{m}_t$$

Evaluating gradients



Backpropagation

$$\frac{\partial \mathcal{L}}{\partial \theta_l} = \frac{\partial \mathcal{L}}{\partial z_L} \frac{\partial z_L}{\partial z_{L-1}} \cdots \frac{\partial z_l}{\partial \theta_l}$$



Tips & tricks

Kernel trick

- Non-linear patterns in the data
- Project data into a higher-dimensional space

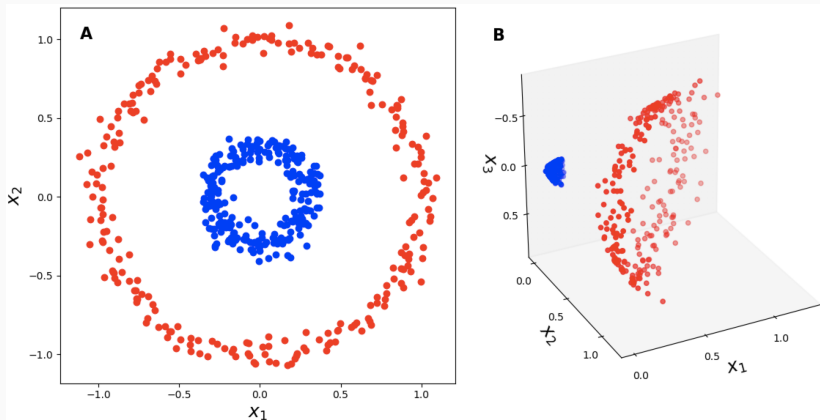


Figure 2: Figure by Gregory Gundersen

- Change LR through training
- Typically: Quick warm up to target, then decay to zero
- Improved convergence

Listing 8 Python

```
schedule = keras.optimizers.schedules.CosineDecay(  
    initial_learning_rate=1e-3  
)  
optimizer = keras.optimizers.Adam(learning_rate=schedule)
```

Gradients can become excessively large or vanishingly small

Exploding gradients

- Unstable training (jumping erratically)
- Numerical issues (overflow)

Remedies

- Batch normalization
- Gradient clipping

Vanishing gradients

- Slow training (barely moving)
- Numerical issues (underflow)

Remedies

- Batch normalization
- Different activation functions
- Residual connections

- Keep output close to mean 0 and variance 1

Listing 9 Python

```
1 network = keras.models.Sequential([
2     keras.Input((3,)),
3     keras.layers.Dense(4, activation="relu"),
4     keras.layers.BatchNormalization(),
5     keras.layers.Dense(4),
6     keras.layers.BatchNormalization(),
7     keras.layers.Activation("relu"),
8     keras.layers.Dense(2, activation="softmax")
9 ])
```

- Keep output close to mean 0 and variance 1

Listing 10 Python

```
1 network = keras.models.Sequential([
2     keras.Input((3,)),
3     keras.layers.Dense(4, activation="relu"),
4     keras.layers.BatchNormalization(),
5     keras.layers.Dense(4),
6     keras.layers.BatchNormalization(),
7     keras.layers.Activation("relu"),
8     keras.layers.Dense(2, activation="softmax")
9 ])
```

Gradient clipping

Scale down gradients if they exceed certain threshold

Value clipping

- Restrict gradients to a specified range
- Each gradient clipped individually

Norm clipping

- Restrict the size (norm) of the gradient to a specified range
- All gradients rescaled so that the norm becomes smaller

Listing 11 Python

```
optimizer=keras.optimizers.Adam(  
    learning_rate=1e-3,  
    clipvalue=0.5,  
    clipnorm=1.0)
```

Residual / skip connections

- Add output of a layer with its input
- Removes vanishing gradients
- Layer learns the “residual”: $f(x) = r(x) + x$

Listing 12 Python

```
inputs = keras.Input(shape=(64,))

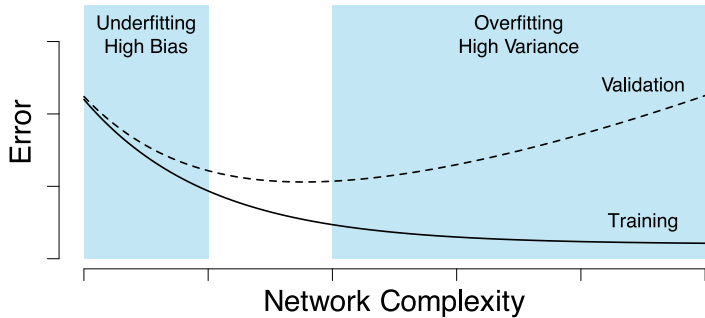
residual = keras.layers.Dense(64)(inputs)
outputs = keras.layers.Add()([residual, inputs])
outputs = keras.layers.Activation("relu")(outputs)

outputs = keras.layers.Dense(10, activation="softmax")(outputs)

model = keras.Model(inputs, outputs)
```

Guards against overfitting

Large networks tend to overfit

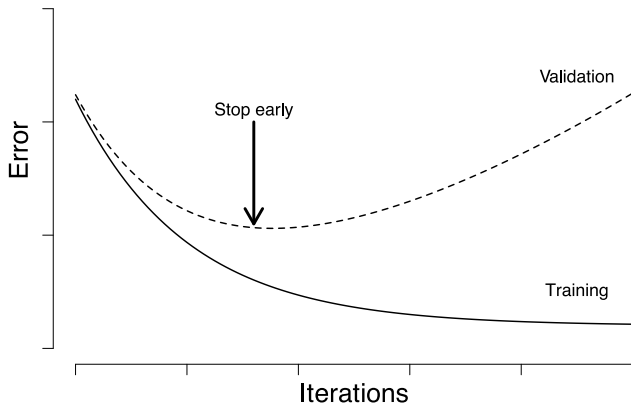


Large networks tend to overfit

Remedies

- Early stopping
- Regularization
- Dropout
- Add more data
- ...

Early stopping



Listing 13 Python

```
early_stopping = keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    restore_best_weights=True  
)  
  
model.fit(  
    X_train, y_train,  
    validation_data=(X_val, y_val),  
    epochs=100,  
    callbacks=[early_stopping]  
)
```

Add (weighted) norm of the parameters to the loss

$$\mathcal{L}(x, \theta) = \mathcal{L}_0(x, \theta) + \lambda \|\theta\|$$

L1

- $\|\theta\|_1 = \sum |\theta|$
- Encourages sparse weights
- Discourages large weights
- Feature selection/pruning

L2

- $\|\theta\|_2 = \sqrt{\sum \theta^2}$
- Encourages spread out weights
- Discourages large weights

Listing 14 Python

```
network = keras.Sequential([
    keras.layers.Dense(64,
        activation="relu",
        kernel_regularizer=keras.regularizers.l1(0.01)),
    keras.layers.Dense(32,
        activation="relu",
        kernel_regularizer=keras.regularizers.l2(0.02)),
    keras.layers.Dense(10, activation="softmax")
])
```

- In keras
 - `kernel_regularizer`: Weights
 - `bias_regularizer`: Bias
 - `activity_regularizer`: Layer output

Dropout

During training, “turn off” each activation with a probability p

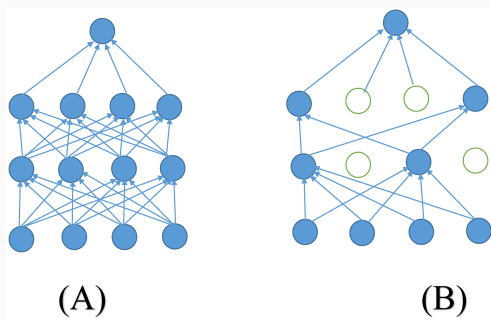


Figure 3: Visualization of dropout

- Better generalization
- Reduced dependence on single neurons
- Reduced expressiveness
- Increased variance during training

Listing 15 Python

```
1 network = keras.Sequential([
2     keras.Input((2,)),
3     keras.layers.Dense(64, activation="relu"),
4     keras.layers.Dropout(0.1),
5     keras.layers.Dense(64, activation="relu"),
6     keras.layers.Dropout(0.05),
7     keras.layers.Dense(10, activation="softmax")
8 ])
9
10 x = keras.random.normal((100, 2))
11
12 network(x)
13 network(x, training=True)
```

Listing 16 Python

```
1 network = keras.Sequential([
2     keras.Input((2,)),
3     keras.layers.Dense(64, activation="relu"),
4     keras.layers.Dropout(0.1),
5     keras.layers.Dense(64, activation="relu"),
6     keras.layers.Dropout(0.05),
7     keras.layers.Dense(10, activation="softmax")
8 ])
9
10 x = keras.random.normal((100, 2))
11
12 network(x)
13 network(x, training=True)
```

Special neural architectures

Pros

- Conceptually simple, universal function approximator
- Easy to train, established
- Almost zero assumptions about data

Cons

- Inefficient in high dimensions (many parameters)
- Works only with fixed size input/output
- Almost zero assumptions about data

Assumptions: Data types

Examples:

- Pictures
- Sequences (text, time-series)
- Sets

→ leverage properties of data to our advantage by building networks that make correct assumptions

Recurrent neural network (RNN)

- Works for sequences of different lengths
- Maintain a hidden state $h_t = \sigma_h(W_h * h_{t-1} + W_x x_t + b_h)$
- Output depends on hidden state $y_t = \sigma_y(W_g * h_t + b_y)$

Issues

- Sequential updating
- Limited long-term memory
- Vanishing gradient

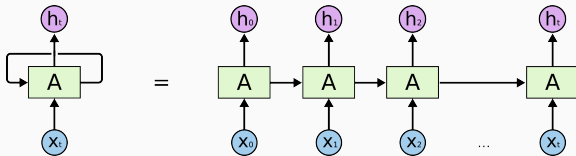


Figure 4: Source: Christopher Olah's blog

Long short-term memory (LSTM)

- Learn to what to “forget” (forget gate) and what to “remember” (input gate)
- Cell state can carry over long term dependencies

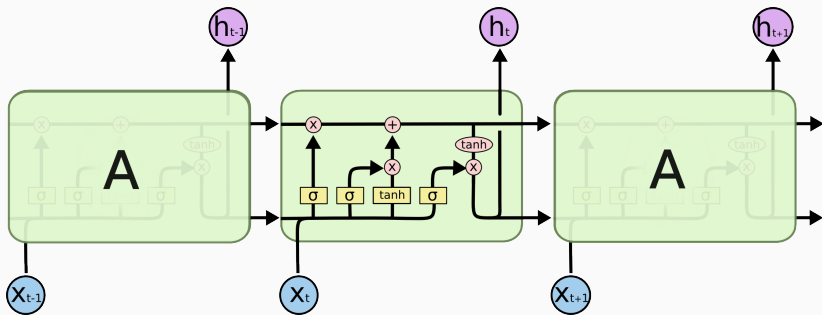


Figure 5: Source: Christopher Olah's blog

- Sequential updating is slow
- Limited memory (even for LSTM)

Solution

- Use positional encoding (“concatenate with time variable”)
- Paralellize the whole computation
- “Attention”: Focus on the relevant parts of the sentence.

- Query: $Q = XW_Q$
- Key: $K = XW_K$
- Value: $V = XW_V$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T)V$$

The old cat sat on the pillow
 $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 $\vec{K}_1$ $\vec{K}_2$ $\vec{K}_3$ $\vec{K}_4$ $\vec{K}_5$ $\vec{K}_6$ $\vec{K}_7$

The $\rightarrow \vec{Q}_1$ $\vec{Q}_1 \cdot \vec{K}_1$ $\vec{Q}_1 \cdot \vec{K}_2$ $\vec{Q}_1 \cdot \vec{K}_3$ $\vec{Q}_1 \cdot \vec{K}_4$ $\vec{Q}_1 \cdot \vec{K}_5$ $\vec{Q}_1 \cdot \vec{K}_6$ $\vec{Q}_1 \cdot \vec{K}_7 \rightarrow \text{softmax}$

old $\rightarrow \vec{Q}_2$ $\vec{Q}_2 \cdot \vec{K}_1$ $\vec{Q}_2 \cdot \vec{K}_2$ $\vec{Q}_2 \cdot \vec{K}_3$ $\vec{Q}_2 \cdot \vec{K}_4$ $\vec{Q}_2 \cdot \vec{K}_5$ $\vec{Q}_2 \cdot \vec{K}_6$ $\vec{Q}_2 \cdot \vec{K}_7 \rightarrow \text{softmax}$

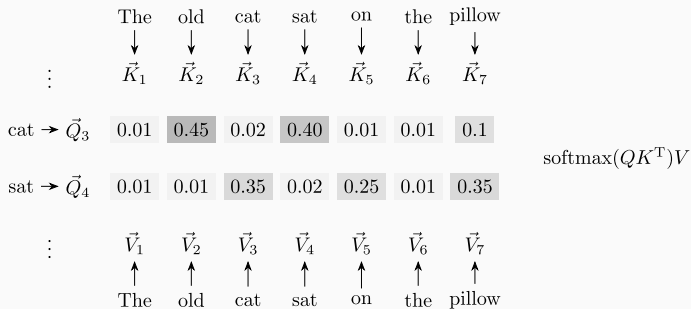
cat $\rightarrow \vec{Q}_3$ $\vec{Q}_3 \cdot \vec{K}_1$ $\vec{Q}_3 \cdot \vec{K}_2$ $\vec{Q}_3 \cdot \vec{K}_3$ $\vec{Q}_3 \cdot \vec{K}_4$ $\vec{Q}_3 \cdot \vec{K}_5$ $\vec{Q}_3 \cdot \vec{K}_6$ $\vec{Q}_3 \cdot \vec{K}_7 \rightarrow \text{softmax}$

sat $\rightarrow \vec{Q}_4$ $\vec{Q}_4 \cdot \vec{K}_1$ $\vec{Q}_4 \cdot \vec{K}_2$ $\vec{Q}_4 \cdot \vec{K}_3$ $\vec{Q}_4 \cdot \vec{K}_4$ $\vec{Q}_4 \cdot \vec{K}_5$ $\vec{Q}_4 \cdot \vec{K}_6$ $\vec{Q}_4 \cdot \vec{K}_7 \rightarrow \text{softmax}$

on $\rightarrow \vec{Q}_5$ $\vec{Q}_5 \cdot \vec{K}_1$ $\vec{Q}_5 \cdot \vec{K}_2$ $\vec{Q}_5 \cdot \vec{K}_3$ $\vec{Q}_5 \cdot \vec{K}_4$ $\vec{Q}_5 \cdot \vec{K}_5$ $\vec{Q}_5 \cdot \vec{K}_6$ $\vec{Q}_5 \cdot \vec{K}_7 \rightarrow \text{softmax}$

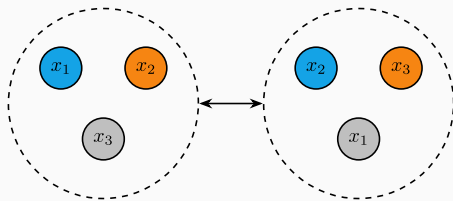
the $\rightarrow \vec{Q}_6$ $\vec{Q}_6 \cdot \vec{K}_1$ $\vec{Q}_6 \cdot \vec{K}_2$ $\vec{Q}_6 \cdot \vec{K}_3$ $\vec{Q}_6 \cdot \vec{K}_4$ $\vec{Q}_6 \cdot \vec{K}_5$ $\vec{Q}_6 \cdot \vec{K}_6$ $\vec{Q}_6 \cdot \vec{K}_7 \rightarrow \text{softmax}$

pillow $\rightarrow \vec{Q}_7$ $\vec{Q}_7 \cdot \vec{K}_1$ $\vec{Q}_7 \cdot \vec{K}_2$ $\vec{Q}_7 \cdot \vec{K}_3$ $\vec{Q}_7 \cdot \vec{K}_4$ $\vec{Q}_7 \cdot \vec{K}_5$ $\vec{Q}_7 \cdot \vec{K}_6$ $\vec{Q}_7 \cdot \vec{K}_7 \rightarrow \text{softmax}$



- Cross-attention
 - Keys and queries are computed from different sources
 - e.g., original (keys) and translated (queries) text
- Multihead attention
 - Multiple attention blocks in parallel
 - Each block “attends” to different representations
- Transformers: Multiple layers of Multihead attention layers and MLP

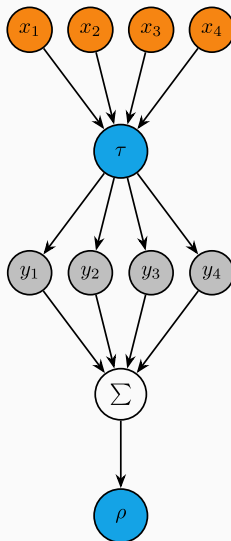
- What if we do not have a fixed order?
- Instead, we have sets

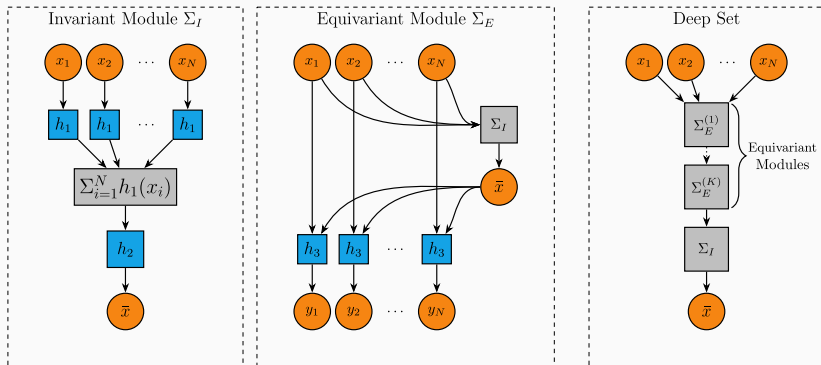


- Permutation invariant function: $f(x) = f(\pi(x))$
- Embeddings of sets
 - Handle different set sizes
 - Permutation invariant
 - Interactions between elements

$$f(X = \{x_i\}) = \rho(\sigma(\tau(X)))$$

- τ : Permutation equivariant function
- σ : Permutation invariant pooling function (sum, mean)
- ρ : Any function





Examples & further references

- Keras code examples
- Tensorflow playground
- Chollet (2021), GitHub
- Urban & Gates (2021)

- Chollet, F. (2021). *Deep learning with Python*. Manning Publications.
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv Preprint arXiv:1412.6980*.
- Kunc, V., & Kléma, J. (2024). Three decades of activations: A comprehensive survey of 400 activation functions for neural networks. *arXiv Preprint arXiv:2402.09092*.
- Shahab, O., El Kurdi, B., Shaukat, A., Nadkarni, G., & Soroush, A. (2024). Large language models: A primer and gastroenterology applications. *Therapeutic Advances in Gastroenterology*, 17, 17562848241227031.
- Urban, C. J., & Gates, K. M. (2021). Deep learning: A primer for psychologists. *Psychological Methods*, 26(6), 743.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep Sets. *Advances in Neural Information Processing Systems*, 30.